CancellationTokens in C#

A practical reference for cooperative cancellation, with the ASP.NET Core and Blazor Server patterns that actually matter in production.

1. What a CancellationToken actually is

A **CancellationToken** is a small, cheap struct that carries a single piece of information through your call chain: *someone wants this operation stopped*. It is the read-only half of a pair:

- **CancellationTokenSource (CTS)** — the producer. You create it, you call `Cancel()`, you dispose it. It owns the signal.
- **CancellationToken** — the consumer view you hand to methods. It can *observe* the signal but can never trigger it.

```
using var cts = new CancellationTokenSource();

Task work = DoWorkAsync(cts.Token);   // hand out the read-only view
cts.Cancel();                          // only the source can pull the trigger
```

The token is a struct — passing it around costs essentially nothing.

2. Cancellation is cooperative

This is the single most important idea, and the one that trips people up. Nothing is killed, aborted, or forcibly unwound. `Cancel()` simply flips a flag and runs any registered callbacks. It is entirely on the code holding the token to *notice* and bail out.

> **The consequence**
> A tight `while (true)` loop that never inspects its token will run forever, no matter how many times you call `Cancel()`. Accepting a token is a promise to check it.

3. Observing the token

Three idioms, chosen by whether cancelling is exceptional, expected, or needs to interrupt something that isn't natively cancellable.

Throw — the default in async chains

Use this when cancellation should unwind the whole operation. It is what every framework method does internally.

```
token.ThrowIfCancellationRequested();   // raises OperationCanceledException
```

Poll — when you want to exit gracefully

Use this in loops where you'd rather finish cleanly, flush a buffer, or return partial work instead of throwing.

```
while (!token.IsCancellationRequested)
{
    ProcessNextBatch();
}
return partialResults;   // graceful stop, no exception
```

Register — to unblock something that can't be cancelled

A callback fires the moment cancellation is requested. The classic use is forcing a blocking resource to fail fast.

```
using var registration = token.Register(() => socket.Close());
await socket.ReceiveBlockingAsync();   // now interruptible
```

Dispose the registration — otherwise the callback stays attached to the token's lifetime.

4. Where tokens come from in ASP.NET Core

In web code you rarely construct a CTS yourself. The framework creates one and hands you the token — you just have to accept it and pass it down.

```
app.MapGet("/orders", async (AppDbContext db, CancellationToken ct) =>
    await db.Orders.Where(o => o.IsOpen).ToListAsync(ct));
```

Minimal APIs and MVC bind CancellationToken parameters automatically.

That token is `HttpContext.RequestAborted`. When the user closes the tab or the connection drops, it fires — and if you threaded it into EF Core, ADO.NET or `HttpClient`, the in-flight work stops instead of burning a thread-pool thread and a SQL connection on a response that nobody will ever read.

> **Why this matters for SQL Server**
>
> `SqlCommand.ExecuteReaderAsync(ct)` doesn't merely abandon the task locally — it sends an *attention signal* to SQL Server, aborting the running query on the server. On expensive reports that is real, measurable resource savings, not just tidier code.

Other tokens the host gives you:

- `IHostedService.StartAsync / StopAsync` — startup and graceful-shutdown tokens. The stop token respects `ShutdownTimeout` (5 seconds by default).
- `BackgroundService.ExecuteAsync(stoppingToken)` — the loop token for long-running workers.
- `IHostApplicationLifetime.ApplicationStopping` — injectable anywhere you need shutdown awareness.

5. Blazor Server: the gap you have to fill yourself

ComponentBase does not give you a cancellation token, and `RequestAborted` is meaningless once the circuit is established — the HTTP request that served the page is long gone. If a user navigates away mid-load, your `OnInitializedAsync` continuation still resumes and calls `StateHasChanged()` on a disposed component.

The fix is to own a CTS at the component level and cancel it on disposal.

```
@implements IDisposable

@code {
    private readonly CancellationTokenSource _cts = new();
    private List<Order>? _orders;

    protected override async Task OnInitializedAsync()
    {
        using var db = await DbFactory.CreateDbContextAsync(_cts.Token);
        _orders = await db.Orders.ToListAsync(_cts.Token);
    }

    public void Dispose()
    {
        _cts.Cancel();
        _cts.Dispose();
    }
}
```

The component-scoped CTS is the Blazor Server equivalent of RequestAborted.

Debouncing a MudBlazor search field

The same pattern kills stale queries: cancel the previous CTS on every keystroke so only the latest request survives to touch the database.

```
private CancellationTokenSource? _searchCts;

private async Task OnSearchChanged(string term)
{
    _searchCts?.Cancel();
    _searchCts?.Dispose();
    _searchCts = new CancellationTokenSource();
    var token = _searchCts.Token;

    try
    {
        await Task.Delay(300, token);          // debounce window
        _results = await Service.SearchAsync(term, token);
        await InvokeAsync(StateHasChanged);
    }
    catch (OperationCanceledException)
    {
        // superseded by a newer keystroke — nothing to do
    }
}
```

Note InvokeAsync(StateHasChanged) — the continuation may not be on the renderer's sync context.

6. Timeouts and linked tokens

A CTS can cancel itself on a timer, and several tokens can be combined so that *any* of them cancels the result. This is how you express "stop if the user leaves, **or** the component is disposed, **or** thirty seconds elapse".

```
using var timeout = new CancellationTokenSource(TimeSpan.FromSeconds(30));

using var linked = CancellationTokenSource.CreateLinkedTokenSource(
    requestToken,      // client disconnected
    timeout.Token,     // took too long
    _cts.Token);       // component disposed

await LongRunningQueryAsync(linked.Token);
```

CancelAfter() can also arm or re-arm a timer on an existing source.

> **Dispose linked sources**
> A linked CTS registers callbacks on each of its parent tokens. If you don't dispose it, those registrations survive for as long as the parents do. In a Blazor circuit — where parent tokens can live for hours — this is the one genuine leak scenario with tokens.

7. Handling the exception

```
try
{
    await DoWorkAsync(ct);
}
catch (OperationCanceledException) when (ct.IsCancellationRequested)
{
    Logger.LogDebug("Operation cancelled by the caller.");
    // expected — do not surface this as a 500
}
```

TaskCanceledException derives from OperationCanceledException, so catch the base type.

The `when` filter is doing real work here. It separates "we cancelled on purpose" from an unrelated timeout bubbling up — `HttpClient`'s internal 100-second timeout also surfaces as a `TaskCanceledException`, and that one *is* a failure you want to see in your logs.

8. Rules of thumb

- If a method is `async` and performs I/O, it should take a token and pass it down — every layer, all the way to the driver. A token that stops halfway does nothing.
- Pass `CancellationToken.None` explicitly when you genuinely don't want cancellation. It documents the intent far better than an overload that silently drops the token.
- Never swallow `OperationCanceledException` and return a default value — the caller will read that as success.
- Think before cancelling a `SaveChangesAsync`. Inside an explicit transaction the rollback is clean; without one, partial-write semantics are murkier than people assume.
- Put the token last in the parameter list, by convention, and name it `cancellationToken` in public APIs — ASP.NET Core's model binder and most analysers expect it.
- Dispose every CTS you create, and always dispose linked sources.

9. Quick reference

Member	What it does
`cts.Cancel()`	Signals the token. Runs registered callbacks synchronously on the calling thread.
`cts.CancelAsync()`	.NET 8+. Runs callbacks off the caller's thread — safer under a lock.
`cts.CancelAfter(delay)`	Arms a timer that cancels the source automatically.
`new CancellationTokenSource(delay)`	Same as above, set at construction. The usual way to express a timeout.
`token.IsCancellationRequested`	Polls the flag. Returns bool, never throws.
`token.ThrowIfCancellationRequested()`	Throws OperationCanceledException if signalled.
`token.Register(callback)`	Runs a callback on cancellation. Dispose the returned registration.
`token.CanBeCanceled`	False for CancellationToken.None — lets you skip setup work.
`CancellationToken.None`	A token that will never be cancelled. Explicit opt-out.
`CreateLinkedTokenSource(a, b)`	Cancels when any source token cancels. Must be disposed.
`HttpContext.RequestAborted`	Fires when the HTTP client disconnects. Not useful in a Blazor circuit.

10. Where to read more

- **Cancellation in managed threads** — the conceptual overview of the cooperative model, on Microsoft Learn under `dotnet/standard/threading`.
- **CancellationToken & CancellationTokenSource API reference** — the full member list, including the .NET 8 additions.
- **ASP.NET Core Blazor component lifecycle** — the official guidance on disposal and long-running work in components, which is where the component-scoped CTS pattern comes from.

One sentence to remember: a CancellationToken is a request, not a command — it only works if every layer of your code agrees to listen.